Cellular Automata: Conway's Game of Life

By June Walenciak

Abstract

This report covers the integration of Conway's Game of Life in python using *matplotlib* for display and handling inputs. A major part of this project was to have settings for the cellular automata to modulate the behavior of the cellular automata. Some of the settings were dimensions and the thresholds for death and reproduction. A minor goal was to optimize Conway's Game of Life using a differential array to limit the search area. The project was overall a success and accomplished all of its major and most of its minor goals.

Introduction

Although Conway's Game of Life has origins that predate the 1960s, it was publicly published in October 1970 by John Conway in *Scientific American*. The purpose of Conway's Game was to make a program that would satisfy three criteria laid out by Martin Gardner in the 1970 paper:

> "1. There should be no initial pattern for which there is a simple proof that the population can grow without limit.
>
> 2. There should be initial patterns that apparently do grow without limit.
>
> 3. There should be simple initial patterns that grow and change for a considerable period of time before coming to end in three possible ways: fading away completely (from overcrowding or becoming too sparse), settling into a stable configuration that remains unchanged thereafter, or entering an oscillating phase in which they repeat an endless cycle of two or more periods."
>
> -Martin Gardner [1]

I think to reduce the purpose of Conway's Game of Life down to a single simple criteria it would be to make a system that was relatively simple, but also intellectually captivating for mathematicians to chew on. This is why I chose this project, because it is a simple project to

integrate, it has interesting optimization problems when it is done because it can be scaled up, and it is an icon of mathematics.

Design

In order to fulfill the goals of this project I needed to create a settings file that stored settings that could be changed through it. For that file I chose it to be a JSON file because of the corresponding python library and the dictionary it creates for the JSON members. The variables in the JSON file are: height, width, radius_death (the radius used for live cells when calculating adjacencies), radius_generate (the radius for dead cells when calculating adjacencies), underpop_tsh (underpopulation threshold), overpop_tsh (overpopulation threshold), and reproduce_adj (the number of adjacent cells required to reproduce).

Conway's Game of Life hinges around a single function, the adjacency function, the function that counts how many living cells are adjacent to a cell. Once this is created the rest of the project is easy. So I created a function called "cell_adjacencies" that takes the input of the map (the board), the radius, and the cell coordinates. It uses the radius to find the edges of a rectangle of the function will consider adjacent and turns it into a new matrix. The cell_adjacencies function then runs through the matrix and for all values that are true it adds one to the adjacent_cells integer variable. Once it has run through all of the adjacent cells it subtracts one from the adjacent_cells value if the origin cell is alive, because it should not count towards the adjacent_cells. Finally it returns adjacent_cells.

The obvious next step was to integrate a function that iterates the game forward a single time unit. It does that by running a loop through every cell on the board and checking whether a cell should die or live using the cell adjacency function. This gets majorly optimized later...

Later I optimized the game iteration function. I had noticed that with a radius of one each cell, each iteration was being was being ran through 9 times. If the radius was two they were ran through 25 times. If the radius was three they were ran through 36 times each. Most of those cells would usually be unchanging white space. In order to optimize this I decided to store the previous iteration of the map and make something called the "differential map" which would compare the current iteration of the game map and the previous and any elements that were not equal would be true in the differential map, and false if they were equal. The differential map measures the change of the matrix. From the differential map, for each true element a radius (equal to the larger of the death and reproduction radius) is drawn around them and set to true and this new matrix is the search matrix. Now when the game iteration function is run through it does not calculate any cell adjacencies unless the search

matrix for that cell is true, because it is impossible for that cell to have changed if that element's corresponding cell in the search matrix is false.

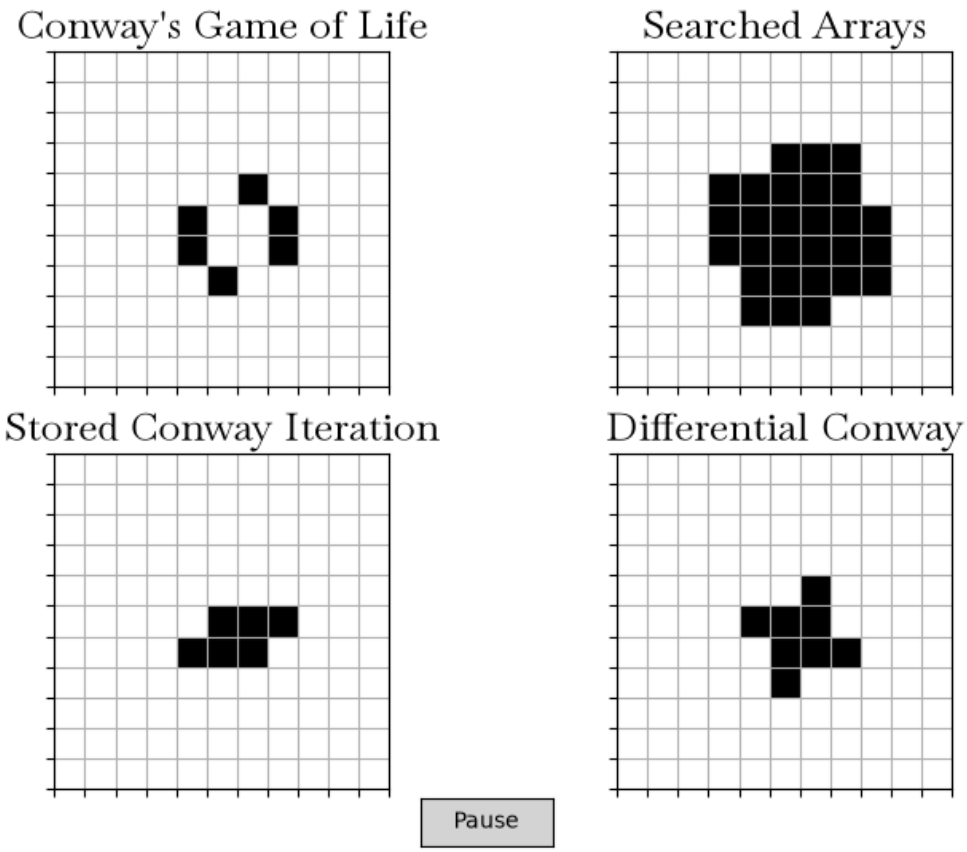


Figure 1: Output Example of My Conway's Game

I also made global variables to track whether the game is paused, a pause button that switches that variable, functionality to click to the diagram while paused to add/remove cells, and a boolean variable that chooses whether you want to import a CSV file of conway coordinates.

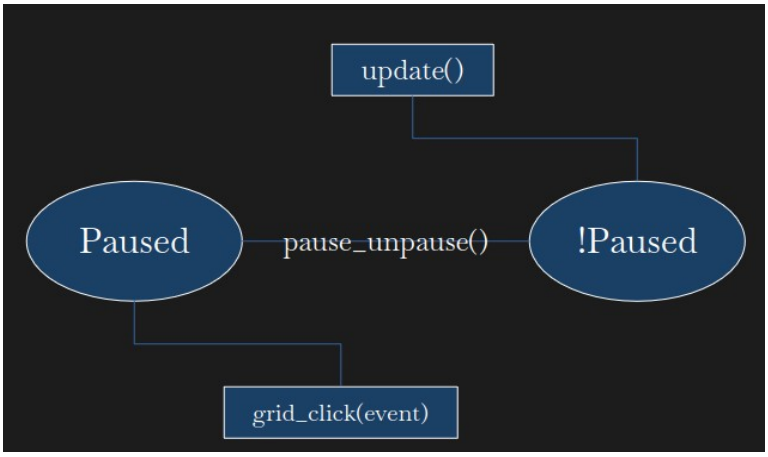


Figure 2: Finite State Machine (very sparse, only 2 states)

Result

The project was successful in achieving all of its goals. The optimization I did wasn't as effective as I would have hoped, but I think it is more optimized than the original state. I learned how to make buttons in matplotlib, I learned generally more about formatting in matplotlib like having custom fonts, I got more comfortable with having file selection with tkinter, and I also learned how to refresh multiple subplots at once. If I were to work on this project further I think I would try to optimize it more, My initial stretch goal was for it to be able to handle a 4000x4000 array, but it was slow with even 1000x1000. I would also be interested in making some sort of cellular automata that mimics plants rather than bacteria, something that could generate stretching limbs, roots, and leaves.

Conclusion

The project overall was a success. If I were to do it all again I would have used object-oriented programming more. When I started this project I had a narrow view of how expansive and overwhelming the variables would get and I wished I had started from the beginning by making classes that define the boards and their behaviors and such.

References

[1] M. Gardner. “MATHEMATICAL GAMES The fantastic combinations of John Conway's new solitaire game ‘life’,” Scientific American, vol. 223. pp 120-123, October 1970. http://drikus.net/exact/automaat/ref/MathematicalGames.pdf Accessed October 9th, 2026.